Bitwise Operators in Java

Bitwise operators perform operations **directly on the binary (bit) representation** of integer data types (`byte`, `short`, `int`, `long`, and `char`). They are widely used in **system programming, embedded systems, networking, cryptography, graphics, and performance optimization**.

Bitwise Operators in Java

Operator	Name	Description	Example
`&`	Bitwise AND	Returns 1 only if both bits are 1	`5 & 3`
`|`	Bitwise OR	Returns 1 if either bit is 1	`5 | 3`
`^`	Bitwise XOR	Returns 1 if bits are different	`5 ^ 3`
`~`	Bitwise NOT	Inverts all bits	`~5`
`<<`	Left Shift	Shifts bits to the left	`5 << 2`
`>>`	Right Shift	Shifts bits to the right (signed)	`20 >> 2`
`>>>`	Unsigned Right Shift	Shifts bits to the right and fills left bits with 0	`-20 >>> 2`

Binary Representation Example

Consider two numbers:

```
A = 5  = 00000101
B = 3  = 00000011
```

---

1. Bitwise AND (`&`)

Returns **1 only if both bits are 1**.

Truth Table

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Example

```
public class BitwiseAND {
    public static void main(String[] args) {

        int a = 5;
        int b = 3;

        System.out.println(a & b);

    }
}
```

Working

```
5 = 0101
3 = 0011
------------
    0001
```

Output

```
1
```

---

2. Bitwise OR (|)

Returns **1 if at least one bit is 1**.

Truth Table

A	B	A | B
0	0	0
0	1	1
1	0	1
1	1	1

Example

```
System.out.println(5 | 3);
```

Working

```
0101
0011
------
0111
```

Output

```
7
```

---

3. Bitwise XOR (^)

Returns **1 when bits are different**.

Truth Table

A	B	A ^ B
0	0	0

A	B	A ^ B
0	1	1
1	0	1
1	1	0

Example

```
System.out.println(5 ^ 3);
```

Working

```
0101
0011
------
0110
```

Output

```
6
```

---

4. Bitwise NOT (~)

Flips every bit.

Example

```
System.out.println(~5);
```

Working

```
5

00000000 00000000 00000000 00000101

After NOT

11111111 11111111 11111111 11111010
```

Output

```
-6
```

Note: Java uses **2's complement** representation for negative numbers.

---

5. Left Shift (<<)

Shifts bits to the left.

Each left shift approximately **multiplies the number by 2**.

Example

```
System.out.println(5 << 2);
```

Working

```
5

00000101

Shift Left by 2

00010100
```

Output

```
20
```

---

6. Right Shift (>>)

Shifts bits to the right.

Each right shift approximately **divides the number by 2**.

Example

```
System.out.println(20 >> 2);
```

Working

```
20

00010100

Shift Right by 2

00000101
```

Output

```
5
```

---

7. Unsigned Right Shift (>>>)

Moves bits to the right and fills the leftmost bits with **0**, regardless of the sign.

Example

```
System.out.println(-20 >>> 2);
```

Output

```
1073741819
```

This operator is mainly used in **low-level programming** and is less common in everyday Java applications.

Summary Table

Expression	Binary Result	Decimal Result
`5 & 3`	`0001`	**1**
`5 | 3`	`0111`	**7**
`5 ^ 3`	`0110`	**6**
`~5`	Inverted Bits	**-6**
`5 << 2`	`10100`	**20**
`20 >> 2`	`00101`	**5**

Significant Uses of Bitwise Operators

1. Checking Even or Odd Numbers

```
int number = 15;

if ((number & 1) == 0)
    System.out.println("Even");
else
    System.out.println("Odd");
```

Why use it?
Checking with `& 1` is very fast because it examines only the least significant bit.

---

2. Swapping Two Numbers Without a Temporary Variable

```
int a = 10;
int b = 20;

a = a ^ b;
b = a ^ b;
a = a ^ b;
```

---

3. Multiplication by Powers of 2

Instead of

```
x * 8
```

use

```
x << 3
```

Example

```
int x = 7;

System.out.println(x << 3);
```

Output

```
56
```

---

4. Division by Powers of 2

Instead of

```
x / 4
```

use

```
x >> 2
```

---

5. Setting a Particular Bit

```
int number = 8;

number = number | (1 << 2);
```

Useful in **permission management** and **flag settings**.

---

6. Clearing a Particular Bit

```
number = number & ~(1 << 2);
```

---

7. Toggling a Bit

```
number = number ^ (1 << 3);
```

8. Checking Whether a Particular Bit Is Set

```
if ((number & (1 << 2)) != 0)
    System.out.println("Bit is set");
```

9. Finding Whether a Number Is a Power of Two

```
public class PowerOfTwo {

    public static void main(String[] args) {

        int n = 16;

        if ((n & (n - 1)) == 0)
            System.out.println("Power of Two");
        else
            System.out.println("Not Power of Two");

    }
}
```

10. Permission Management (Real-World Example)

Many applications store user permissions as **bit flags**.

```
Permission      Binary

Read            0001
Write           0010
Execute         0100
Delete          1000
```

Suppose a user has:

```
Read + Write

0001
0010
------
0011
```

Checking the **Write** permission:

```
if ((permission & 2) != 0)
    System.out.println("Write Permission Granted");
```

This technique is widely used in:

- Operating Systems
- Database Systems
- File Permissions
- Network Protocols

Interview Questions

1. What is the difference between **logical operators** (`&&`, `||`) and **bitwise operators** (`&`, `|`)?
2. Why does `~5` produce `-6` in Java?
3. What is the difference between `>>` and `>>>`?
4. How can you check whether a number is even or odd using a bitwise operator?
5. Write a Java program to swap two numbers using the XOR operator.
6. How can you determine if a number is a power of two using bitwise operations?
7. Explain one real-world use case of bitwise operators in software development.